

OOP & Inheritance

Introduction to Programming

Rae Harbird, r.harbird@ucl.ac.uk

Inheritance

Inheritance

What is inheritance and why do we need it?

- ▶ So far, we have looked at ways of organising code into classes as a means of:
 - ▶ Keeping all the code associated with a particular thing or concept in one place. A class contains the data (instance variables) and logic (methods) that we will use for the objects created from the class.
 - ▶ Hiding the implementation details from the person using the class. This is called encapsulation and allows us to change the implementation more easily at a later time if we need to.
 - ▶ But what can we do if we need classes which are very similar to each other? For example: an `undergraduateStudent` class and a `postgraduateStudent` class?
- ▶ Inheritance can help here.

An example:

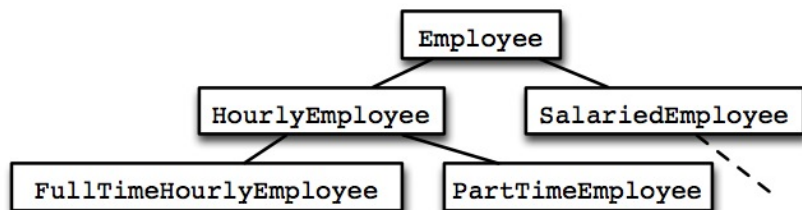


Figure 1: Class diagram

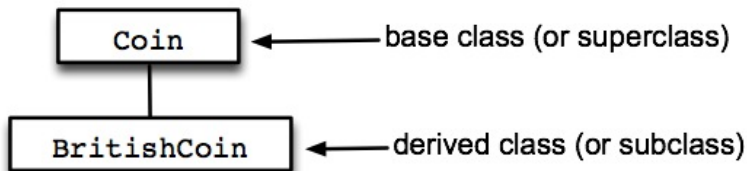
- Inheritance provides a tool for code reuse by deriving one class from another. The derived class automatically inherits the features of the old (base) class and may add features as well.

More detail

- ▶ A derived class object inherits the instance variables, static variables (constants), and methods of the base class, and may add additional instance variables, static variables, and methods of its own.
- ▶ An object of a derived class has the type of the derived class, and it also has the type of the base class, and more generally, has the type of every one of its ancestor classes.

Another example

- ▶ Take a look at the code for `Coin` and `BritishCoin`



- ▶ `BritishCoin` contains extra information specific to British coins, such as each coin's monetary value in pence, and extra methods such as a method displaying the monetary value.

BritishCoin

- ▶ Python knows that `BritishCoin` is derived from `Coin` because of the header of the class definition:

```
class BritishCoin(Coin) :  
    ...
```

Derived class constructor

- ▶ A constructor of a base class can be used in the definition of a constructor for a derived class. The key word `super` is used as a prefix to call the constructor of the base class.
- ▶ By using the base class constructor, you make sure that you have initialised all the attributes belonging to the base class. After you have initialised the base class, you can write the code for the initialisation of the derived class and its attributes.

BritishCoin class

```
from coin import Coin
```

```
class BritishCoin(Coin) :
```

```
    UNIT = "pence"
```

```
    ALLOWED_VALUES = {1 : "one", 2 : "two", 5 : "five", 10 : "ten", \
                        20 : "twenty", 50 : "fifty", 100 : "one hundred", \
                        200 : "two hundred"}
```

```
    def __init__(self, theValue = 1) :
```

```
        super().__init__()
```

```
        self.value = theValue
```

More of the BritishCoin class

```
@property
def value(self):
    return self._value

@value.setter
def value(self, valueAmount) :
    if valueAmount in list(self.ALLOWED_VALUES) :
        self._value = valueAmount
    else :
        self._value = min(list(self.ALLOWED_VALUES))

def __str__(self) :
    return "value: " + self.ALLOWED_VALUES[self.value] + " " \
        + self.UNIT + " showing face: " + super().__str__()

def __eq__(self, otherCoin) :
    if isinstance(otherCoin, BritishCoin) :
        return (self.sameFaceAs(otherCoin) and
                self.sameValueAs(otherCoin))
    else :
        raise TypeError("Both objects must have the same type.")
```

Overriding and Method Invocations Using `super`

- ▶ A method may be redefined in a derived class so that it performs differently from how it performs in the base class. This is called **overriding** the method definition. [See e.g. the definitions of `__eq__(...)` and `__str__()` in `BritishCoin`.]
- ▶ The definition for an overridden method is given in the class definition of the derived class, in the same way as the definitions of any added methods.
- ▶ The key word `super` can be used to invoke the original (overridden) method in the base class. [See e.g. the definition of `sameFaceAs(...)` in `BritishCoin`.]

Using the main function for testing

- ▶ The short main function in BritishCoin serves to illustrate how overriding works in classes that form part of inheritance hierarchy. Several BritishCoin objects are created and compared.

```
def main() :  
    # a simple main function to test this class  
    coin1 = BritishCoin()  
    coin2 = BritishCoin(1)  
    coin3 = BritishCoin(2)  
    coin4 = BritishCoin(3)  
    print("First coin: " + str(coin1))  
    print("Second coin: " + str(coin2))  
    print("Third coin: " + str(coin3))  
    print("Fourth coin: " + str(coin4))  
    print("The first two coins are ", end="")  
    if coin1 != coin2 :  
        print("not ", end="")  
    print("equal.")
```

Using methods in derived classes

- ▶ You can use any of the superclass methods on objects of a derived class. Take a look at britishcoin_prog.py.

```
from british_coin_module import BritishCoin

def main():

    theBritishCoin = BritishCoin()

    selection = 'a';
    while selection != 'q':
        print("\tType: ")
        print("\t\tf - to flip the coin")
        print("\t\tq - to quit\n")

        selection = input("\tEnter selection: ")
        if selection[0] == 'f':
            theBritishCoin.flip()
            print("\n\tThe result is: " + str(theBritishCoin))

    print("\tPROGRAM ENDED\n")
```

Employee hierarchy

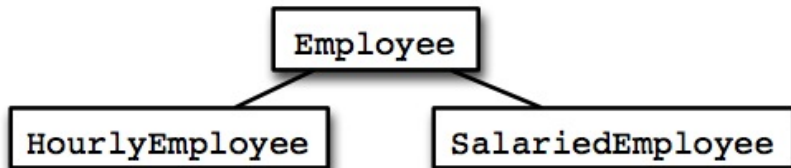


Figure 2: Class diagram

- ▶ The Employee class hierarchy includes the base class Employee and two derived classes HourlyEmployee and SalariedEmployee. These classes illustrate many of the points covered so far in the lecture, including, for example, overriding methods. Here is an example showing how the `__str__()` method in each class is overridden.

Employee classes

► Employee:

```
def __str__(self):  
    return "Name: " + self.name + ", start date: " + str(self.date)
```

► SalariedEmployee:

```
def __str__(self):  
    return super().__str__() + " salary: " + str(self.salary)
```

► HourlyEmployee:

```
def __str__(self):  
    return super().__str__() + " hours per week: "  
        + str(self.hoursPerWeek) \  
        + " hourly rate: " + str(self.hourlyRate)
```

Classes and relational operators

- ▶ We know that we can write an `__eq__()` method for comparing two objects and determining if they are equal.
- ▶ You, the programmer, are in charge of what equality means. For example:

```
def __eq__(self, otherCoin) :  
    if isinstance(otherCoin, BritishCoin) :  
        return (self.sameFaceAs(otherCoin) and \  
                self.sameValueAs(otherCoin))  
    else :  
        raise TypeError("Both objects must have the same type.")  
  
def sameFaceAs(self, otherCoin) :  
    return super().__eq__(otherCoin)  
  
def sameValueAs(self, otherCoin) :  
    return self.value == otherCoin.value
```

For more information about the Python object methods which you can redefine, see **Section 3.3 of the documentation**